

SWARMOS — Frontend & UI/UX Ownership Brief

SIH26123 · Edge-AI Distributed Fleet Coordination for AMRs · what I own, what I'm building it with, and what I need from the rest of the team

1. Where my part sits in the system

Per the architecture blueprint, the pipeline runs: User → LLM Module → Task Manager/Coordinator → Multi-Agent AMR Layer → Robot-to-Robot Communication → Server/Cloud ↔ Edge AI Layer ↔ AMR Fleet, feeding into blocks 11–13: Execution Layer, Monitoring Dashboard, and Analytics & Reports.

My scope is blocks 11–13 — the entire operator-facing surface (frontend + UI/UX). Everything upstream — LLM task parsing, task allocation, edge inference, robot-to-robot comms, and the simulation/physics engine itself — belongs to the backend/robotics side. I consume their output; I do not generate it.

This distinction matters for data integrity: my UI visualizes a simulation someone else's backend produces. If a simulation event isn't emitted yet, the UI shows a real "waiting for data" state — never a fabricated number or animation.

2. What I need from the backend team

Confirmed from the stack diagram: **Python + ROS 2 + FastAPI** on the backend, serving **React** on the frontend, with **MySQL** for persistence and **Docker** for deployment. My integration point is one FastAPI service exposing REST endpoints for state/history and a WebSocket for live updates.

Before I build the data layer, I need from whoever owns the simulation/backend:

- The WebSocket event schema — shape of a robot-position update, a task-state change, an incident event
- The REST endpoints for non-realtime data — task list, benchmark runs, analytics aggregates
- Confirmation of what's genuinely available at demo time (so I never have to fake a field)

If this isn't finalized yet, I'll draft a placeholder contract now and hand it over for them to conform to, so I'm not blocked.

3. Tech stack — send this to everyone so nothing is mismatched

Layer	Tool	Why
Framework	React + Vite + TypeScript	Matches the diagram's React frontend; Vite for fast dev loop. No Next.js needed — this is a live ops dashboard, not a public site.
Styling	Tailwind CSS + shadcn/ui (Radix)	Accessible primitives out of the box (dialogs, dropdowns, focus handling) instead of building from scratch.
Motion	Framer Motion	Used only for meaningful motion — route changes, panel transitions, state changes. Not decoration.
Icons	Lucide	One coherent icon system across every screen. No emoji, no mixed icon sets.

Digital-twin renderer	HTML5 Canvas + requestAnimationFrame	50 AMRs is comfortably doable on plain Canvas. Adding PixiJS/Three.js now is unjustified complexity — only add if profiling proves raw Canvas can't hold FPS.
Charts	Recharts	Real data only — every chart bound to an actual metric, never a static image or hard-coded point.
Realtime	Native WebSocket API	Consumes the FastAPI socket directly. Handles reconnect, stale state, and out-of-order events.
State	Zustand	Keeps high-frequency robot position updates out of the main React render tree — critical for 50 moving robots without janky re-renders.
Data fetching	TanStack Query	Cleanly separates slow-changing "server state" (tasks, benchmarks) from fast-changing "realtime state" (positions).

Nothing exotic, nothing that requires backend teammates to change their stack — this only concerns my own layer.

4. Design direction

Per the product brief, generic "dark cards on black with a glowing accent" is explicitly the look to avoid — it reads as templated AI-dashboard, not a credible robotics control product.

Token	Choice	Notes
Background	#15171B / #1C1F24	Charcoal/graphite, never pure black
Accent	#5B7FA6	One restrained desaturated blue — used only for active/selected states, not decoration
Status colors	Green / Amber / Red / Blue	Strictly semantic — reserved for real state, never used decoratively
Type	Inter or Geist	Tabular numerals for all IDs, battery %, latency, timestamps
Surfaces	Flat, bordered panels	No glassmorphism, no glow, modest border-radius
Hero	The warehouse floor itself	A real navigable top-down facility (racks, aisles, chargers, docks) — not a background image

5. Non-negotiables carried over from the master spec

- **No fake anything** — every KPI, chart, status and incident must trace to a real data source, or the UI shows "Unavailable / No Data / Waiting for Data"
- **Robots move by interpolation between real state updates** — no teleporting, no decorative looping animation disconnected from actual state
- **Same entity, same object everywhere** — a robot selected on the map is the same robot object in Fleet, Task Inspector, Incident Inspector and Replay
- **High-frequency state stays out of slow UI** — robot position updates must never force the whole React tree (sidebars, tables, panels) to re-render
- **Exception-first, not alarm-fatigue** — related incidents get grouped ("7 robots affected in Zone C"), not shown as seven separate alerts

6. Build order (P0 first)

#	Stage	What it covers
1	App shell	Three-pane layout — left nav, center workspace, right contextual inspector. Routing for all 13 screens.
2	Live Warehouse canvas	Top-down facility render + robot primitive with real body/heading — not dots. This is the hero screen and gets built first.
3	Realtime data layer	WebSocket contract, Zustand store split into visual/render state vs. slow UI state, reconnect + stale-state handling.
4	Robot inspector	Right-side drawer wired to real selection — identity, health, mission, route, nearby robots, events, decision trace.
5	Fleet table + Incidents	Searchable/filterable roster; exception-first incident center with grouping and drill-down.
6	Simulation Lab, Replay, Benchmarks, Analytics	P1 screens, built once the P0 live-data path is proven end-to-end.

Prepared for the SWARMOS team · frontend & UI/UX ownership brief · references the SIH26123 architecture blueprint and the SWARMOS master UI/UX specification.